

Daary Backend – Detailed Technical Report

1. Project overview and business scope

This repository is a Django REST API backend for a real-estate marketplace and developer platform. It is designed to support both customer-side browsing and developer-side management. The platform revolves around property discovery, buyer authentication, lead capture, and developer project operations.

Business purpose

The application allows:

- buyers to create accounts with phone-based OTP login
- users to browse listings and read detailed property information
- property search with filtering and ranking
- saving favorite properties
- submitting inquiries about properties
- developers to manage their accounts and team members
- developers to advertise and manage projects
- the marketing team to collect leads and export them
- internal dashboards to view aggregated stats
- AI-powered survey-based recommendation matching

Main product domains

The system is divided into these production areas:

1. Buyer auth and identity
2. Property catalog and media
3. Search and ranking
4. Favorite management
5. Inquiry and lead flows
6. Developer account and team management
7. Project management
8. Location hierarchy
9. Marketing lead collection
10. Dashboard analytics
11. AI recommendation engine

What the project creates

The project produces a real-estate backend that connects users to property data and developer operations. It is not only a database; it is an API system that supports end-user authentication, listing retrieval, filtering, recommendations, and internal team workflows.

High-level flow

The platform works as a pipeline:

- buyer enters phone number
- challenge is requested from Akedly
- OTP is sent and verified
- JWT tokens are issued
- buyer browses and searches listings
- buyer saves favorites and sends inquiries
- developer manages project and marketing data
- dashboard and AI modules read from the same core datasets

Business user journeys

Buyer journey

1. User signs up with a phone number.
2. System requests Akedly challenge.
3. OTP is sent to that number.
4. OTP is verified and user becomes active.
5. User queries property listings.
6. User filters by location, type, price, and amenities.
7. User clicks a property and reads details.
8. User can favorite properties or send an inquiry.
9. Inquiry is validated and stored.
10. Admin or agent receives a notification after async task processing.

Developer journey

1. Developer registers account.
2. Developer logs in using email/password.
3. Developer creates or updates account profile.
4. Developer invites team members and assigns permissions.
5. Developer creates a project with details, media, and location.
6. Developer sees dashboard metrics.
7. Developer reviews leads and exports them.

Core technical stack

- Python
- Django
- Django REST Framework
- PostgreSQL-style relational models (SQLite in dev)
- Celery for async background jobs
- JWT for authentication
- custom common validators, permissions, pagination, renderers
- Akedly OTP integration
- custom AI scoring logic

2. Full file structure and file-by-file explanation

Root-level files

manage.py

Purpose: Django project entry point.

What it does:

- loads the project's settings module
- executes management commands such as runserver, migrate, shell, seed, test
- is the standard entry for all Django administrative actions

conftest.py

Purpose: pytest test configuration and shared fixtures.

What it does:

- configures the Django test environment
- sets settings for testing
- provides reusable fixtures for API clients and auth states

requirements.txt

Purpose: Python dependency declaration.

What it includes:

- Django
- djangorestframework
- simplejwt
- celery
- faker or test libs
- other project packages

docker-compose.yml

Purpose: defines local services for development.

What it does:

- starts backend and supporting containers
- may include Redis, Celery worker, PostgreSQL or other services

Dockerfile

Purpose: container image for the backend service.

What it does:

- installs dependencies
- copies source code
- configures app startup command

pytest.ini

Purpose: Pytest settings.

What it does:

- sets Django settings for tests
- controls test discovery and markers

Root Django project layer

daary_backend/urls.py

Purpose: top-level API routing.

What it does:

- includes app URL modules
- defines the root path routing
- exposes health, admin, docs, and API routes

daary_backend/celery.py

Purpose: Celery app setup.

What it does:

- initializes Celery with the Django project config
- allows background task scheduling and execution

daary_backend/settings/base.py

Purpose: base project settings used across environments.

Important responsibilities:

- installed apps registration
- middleware configuration
- custom user model setup
- database config
- DRF settings
- JWT config
- CORS config
- logging
- Celery broker and result backend
- OpenAPI generation
- Akealy and AI provider integration

daary_backend/settings/dev.py

Purpose: development overrides.

What it configures:

- debug mode
- local SQLite DB
- local CORS behavior
- console email backend
- development token lifetime settings

daary_backend/settings/prod.py

Purpose: production environment configuration.

What it handles:

- production-safe database config
- environment-based deployments
- security-sensitive settings

daary_backend/settings/test.py

Purpose: test configuration.

What it handles:

- fast test database
- settings suitable for CI and local testing

apps/common module

apps/common/models.py

File purpose: shared model base class.

Main class:

- TimestampedModel

What it does:

- adds created_at and updated_at
- ensures consistent timestamp behavior
- provides common ordering logic

apps/common/validators.py

Purpose: reusable validation rules.

Functions:

- validate_egyptian_phone(value)
 - normalizes phone numbers into a valid local pattern
 - accepts common Egyptian mobile formats
 - rejects malformed values
- validate_birthday(value)
 - ensures the date is not in the future
 - enforces minimum age and realistic upper bound

apps/common/permissions.py

Purpose: access control helpers.

Class:

- IsOwner

What it checks:

- whether the request user owns the target object
- handles obj.user or obj.user_id patterns

apps/common/pagination.py

Purpose: consistent DRF pagination for API responses.

Class:

- StandardPagination

What it does:

- sets page size and max page size
- adds count, next, previous, and page metadata
- keeps list responses consistent across apps

apps/common/renderers.py

Purpose: standard JSON output shape.

Class:

- ApiRenderer

What it does:

- ensures every API response follows a consistent format
- adds success/data/message/meta keys

apps/common/exceptions.py

Purpose: custom DRF exception formatting.

Function:

- custom_exception_handler(exc, context)

What it does:

- intercepts validation and permission errors
- normalizes them into structured response output
- makes front-end error handling predictable

apps/common/akedly.py

Purpose: OTP provider integration.

Main classes and functions:

- AkedlyApiError
- _get_credentials()
- _parse_akedly_response(response)
- get_challenge()
- send_otp()
- verify_otp()

What it does:

- reads external provider credentials from settings
- requests a proof-of-work challenge
- sends OTP messages to phones
- verifies OTP codes against the provider
- translates provider errors into project errors

apps/users module

apps/users/models.py

Purpose: custom buyer profile user model.

Main entities:

- UserManager
- User

Function behavior:

- UserManager.create_user() validates phone, sets password, and saves user
- User is a custom auth model using phone as login identifier
- is_active is false until OTP verification succeeds
- username field is phone-based

Important logic:

- the project uses phone verification instead of email-first signup
- the custom user model stores name and birthday
- access control is tied to the user instance

apps/users/selectors.py

Functions:

- get_user_by_phone(phone)
- get_user_by_id(user_id)

Purpose:

- fetch user data quickly and consistently
- hide query logic from views

apps/users/services.py

Function:

- update_user_profile(user, fields)

What it does:

- updates the profile fields allowed for users
- prevents phone modifications
- validates data before saving

apps/users/serializers.py

Classes:

- UserProfileSerializer
- UserUpdateSerializer

Purpose:

- serializes user objects for responses
- validates update payloads for profile editing

apps/users/views.py

Class:

- MeView

Endpoints:

- GET /api/v1/account/me/
- PATCH /api/v1/account/me/

What it does:

- returns the authenticated user profile
- allows profile update

apps/users/urls.py

Purpose: route definition for user account endpoints.

apps/authentication module

apps/authentication/serializers.py

Main classes:

- PowSolutionSerializer
- SendOtpSerializer
- VerifyOtpSerializer
- AuthTokenSerializer

Purpose:

- validate the data required for challenge/OTP verification
- handle JWT payload serialization for login results

apps/authentication/services.py

Main functions:

- get_akedly_challenge()
- send_akedly_otp(...)
- verify_akedly_otp_and_authenticate(...)

Behavior:

- resolves challenge details from the external OTP service
- normalizes the phone number to the expected format
- creates or fetches the applicant user
- marks user as active after verification
- generates refresh and access tokens
- returns authentication metadata

apps/authentication/views.py

Classes:

- OtpRateThrottle
- ChallengeView
- SendOtpView
- VerifyOtpView
- LogoutView

Important logic:

- challenge is created by calling the Akedly API
- OTP send is validated and rate-limited
- OTP verify validates code and redirects to JWT issue flow
- logout blacklists refresh tokens

apps/authentication/urls.py

Routes:

- challenge/
- send-otp/
- verify-otp/
- token/refresh/
- logout/

apps/authentication/tasks.py

Purpose: background auth-related tasks if required by future flows.

apps/properties module

apps/properties/models.py

Main models:

- PropertyType
- CompletionStatus
- Furnishing
- PaymentMethod
- Agent
- Amenity
- Property
- PropertyImage

What they do:

- Property is the core listing entity
- Agent stores the agent assigned to a listing
- Amenity enables category-based features like parking or security
- PropertyImage stores media references for listings

apps/properties/selectors.py

Functions:

- get_property_by_id(pk)
- get_properties_queryset(filters)

Purpose:

- fetch filtered property data cleanly
- keep query logic out of the view layer

apps/properties/services.py

Function:

- increment_view_count(property, request)

What it does:

- counts a property view without inflating totals from duplicate visits
- uses request/session or cache-based deduplication

- prevents excessive counts from the same user or IP in a short time window

apps/properties/serializers.py

Classes:

- AgentSerializer
- PropertyImageSerializer
- PropertyListSerializer
- PropertyDetailSerializer

Purpose:

- convert property and related data to API payloads
- present nested agent and image data cleanly

apps/properties/views.py

Classes:

- PropertyListView
- PropertyDetailView

Endpoints:

- GET /api/v1/properties/
- GET /api/v1/properties//

What they do:

- list properties with filters
- fetch a single listing
- expose search-friendly result payloads

apps/properties/tasks.py

Purpose: scheduled property maintenance tasks.

Main function:

- calculate_location_medians()

What it does:

- computes median prices by location and property type
- used for ranking and search pricing logic

apps/properties/management/commands/seed_data.py

Purpose: seed base data for development/testing.

What it does:

- creates initial records for listings and supporting metadata
- populates data used by demos or local testing

apps/properties/management/commands/seed_properties.py

Purpose: generate property records in bulk.

What it does:

- creates demo content for testing the listing system
- useful for search and ranking validation

apps/search module

apps/search/ranking.py

Functions:

- `_get_segment_median_price(location, property_type)`
- `compute_relevance_score(...)`
- `apply_ordering(...)`

Purpose:

- evaluate how relevant a property is to a given search
- rank listings by strength of match rather than only recency or price

What the scoring considers:

- property recency
- listing completeness
- price competitiveness
- location relevance
- filter matches

apps/search/services.py

Main function:

- `search_properties(filters=None, ordering=None)`

Purpose:

- central search engine for property discovery

Supported filters:

- location
- type
- bedrooms
- bathrooms
- min and max price
- min and max area
- completion status
- furnishing
- amenities
- payment type

Behavior:

- maps Arabic and local synonyms to canonical property categories
- filters the queryset
- applies ranking logic
- returns sorted results

apps/search/views.py

Class:

- `PropertySearchView`

Endpoint:

- GET /api/v1/search/

What it does:

- accepts user query params
- passes them into the search service
- returns paginated results

apps/search/tasks.py

Function:

- refresh_price_medians()

Purpose:

- updates pricing metadata used by ranking
- keeps search relevance consistent as listings change

apps/favorites module

apps/favorites/models.py

Model:

- Favorite

What it does:

- ties a user to a property
- stores the saved property relationship
- prevents duplicates through a uniqueness constraint

apps/favorites/services.py

Function:

- toggle_favorite(user, property)

What it does:

- adds a favorite if it does not exist
- removes it if it already exists
- returns action result and current favorite state

apps/favorites/serializers.py

Class:

- FavoriteSerializer

Purpose:

- serialize saved property entries for API responses

apps/favorites/views.py

Classes:

- FavoriteListView
- FavoriteToggleView

Endpoints:

- GET /api/v1/favorites/

- POST /api/v1/favorites//toggle/

What they do:

- list current favorites for the authenticated user
- toggle favorite state on a property

apps/inquiries module

apps/inquiries/models.py

Purpose: store property inquiries and buyer interest.

Main components:

- InquiryType
- InquiryStatus
- ContactMethod
- Inquiry

Fields include:

- property
- user or guest information
- name
- phone
- message
- preferred contact method
- requirement summary
- status
- rating
- agent notification flag

apps/inquiries/selectors.py

Functions:

- get_user_inquiries(user)
- get_inquiry_by_id(pk, user=None)

Purpose:

- provide clean query access for user-specific and detail-level inquiry reads

apps/inquiries/services.py

Functions:

- create_inquiry(data, user=None, client_ip=None)
- update_lead(inquiry, status=None, rating=None)

What it does:

- validates phone numbers and requested inquiry data
- prevents duplicate spam entries with a cooldown mechanism
- saves the inquiry record
- triggers async notifications

apps/inquiries/tasks.py

Function:

- send_inquiry_notification(self, inquiry_id)

What it does:

- loads linked property and agent details
- formats a message payload
- posts to webhook or message destination
- marks the inquiry as notified

apps/inquiries/views.py

Classes:

- InquiryCreateView
- MyInquiriesListView
- InquiryDetailView

Endpoints:

- POST /api/v1/inquiries/
- GET /api/v1/inquiries/my/
- GET /api/v1/inquiries//

Purpose:

- create and review buyer inquiries

apps/developer_accounts module

apps/developer_accounts/models.py

Models:

- Permission
- DeveloperAccount
- DeveloperUser

Purpose:

- define developer identity, team, and permission structure
- separate developer users from buyer users

Important logic:

- DeveloperUser is an auth model for developer login
- developers can belong to an account and have permissions
- there is a primary account owner concept

apps/developer_accounts/authentication.py

Class:

- DeveloperTokenAuthentication

Purpose:

- authenticate developer endpoints using a custom token strategy

apps/developer_accounts/permissions.py

Classes:

- IsDeveloperAuthenticated
- HasDeveloperPermission

Purpose:

- restrict access based on developer auth and permissions

apps/developer_accounts/selectors.py

Function:

- get_team_members(account)

Purpose:

- fetch team members assigned to a developer account

apps/developer_accounts/services.py

Functions:

- authenticate_developer(email, password)
- update_developer_account(user, data)
- invite_team_member(account, data)
- update_team_member(member, data)
- deactivate_team_member(member)
- delete_developer_account(user)

Purpose:

- manage the developer login and account lifecycle
- create and update team members
- enforce account ownership rules

apps/developer_accounts/views.py

Classes:

- LoginView
- RegisterView
- AccountView
- TeamListView
- TeamCreateView
- TeamDetailView

Endpoints:

- POST /api/v1/developer/login/
- POST /api/v1/developer/register/
- GET /api/v1/developer/account/
- PUT /api/v1/developer/account/
- DELETE /api/v1/developer/account/
- GET /api/v1/developer/team/
- POST /api/v1/developer/team/invite/
- PUT /api/v1/developer/team/

- DELETE /api/v1/developer/team//

apps/projects module

apps/projects/models.py

Purpose: store developer project records.

Models:

- ProjectType
- ProjectStatus
- Project
- ProjectImage

What it does:

- enables developers to create property or development projects
- stores location, title, description, images, and status metadata

apps/projects/selectors.py

Functions:

- get_project_by_id(pk, account)
- get_projects_queryset(account, filters)

Purpose:

- fetch only the developer account's projects and related data

apps/projects/services.py

Functions:

- create_project(account, data, files)
- update_project(project, data, files)
- delete_project(project)

Purpose:

- perform the project lifecycle operations
- attach media and update project metadata

apps/projects/views.py

Classes:

- ProjectListCreateView
- ProjectDetailView
- ProjectStatsView

Endpoints:

- GET /api/v1/developer/projects/
- POST /api/v1/developer/projects/
- GET /api/v1/developer/projects//
- PUT /api/v1/developer/projects//
- DELETE /api/v1/developer/projects//
- GET /api/v1/developer/projects//stats/

Purpose:

- manage project data and summary statistics

apps/locations module

apps/locations/models.py

Model:

- Location

Purpose:

- represent geographic hierarchy such as city, area, district, or compound zone

What it does:

- supports parent-child relationships
- supports filtering by region or neighborhood

apps/locations/views.py

Class:

- LocationListView

Endpoint:

- GET /api/v1/locations/

Purpose:

- return the location hierarchy and filtered location choices

apps/marketing module

apps/marketing/models.py

Model:

- MarketingLead

Purpose:

- store leads collected from marketing or contact flows

Fields include:

- source
- project reference
- contact information
- metadata

apps/marketing/views.py

Classes:

- MarketingLeadCreateView
- DeveloperLeadListView
- DeveloperLeadExportView

Endpoints:

- POST /api/v1/contact/lead/
- GET /api/v1/developer/leads/

- GET /api/v1/developer/leads/export/

Purpose:

- create leads from external contact forms
- review leads in the developer dashboard
- export them in CSV format

apps/dashboard module

apps/dashboard/services.py

Function:

- get_dashboard_stats(account)

Purpose:

- compute analytics for a developer account

Metrics include:

- total projects
- active projects
- leads count
- leads in recent time window
- source distribution
- location median summaries

apps/dashboard/views.py

Class:

- DashboardOverviewView

Endpoint:

- GET /api/v1/developer/dashboard/overview/

Purpose:

- expose dashboard overview metrics to the frontend

apps/ai module

apps/ai/models.py

Models:

- SurveySession
- SurveyQuestion
- SurveyAnswer
- SurveyResult

Purpose:

- capture a survey flow and store the answers and computed matches

apps/ai/consultation_scoring.py

Functions:

- score_homebuyer_property(property_obj, answers)

- `score_investor_property(property_obj, answers)`
- `score_property_for_track(property_obj, track, answers)`

Purpose:

- compute how well a property matches a user profile or purchase track
- supports recommendation scoring based on property metadata and answers

`apps/ai/services.py`

Functions:

- `_ensure_question_bank()`
- `start_survey(user)`
- `_get_session_for_request(request, session_id)`
- `_serialize_matches(session)`
- `answer_survey_question(request, data)`

Purpose:

- initialize the survey flow
- track answers per session
- generate property matches and return recommendation results

`apps/ai/views.py`

Classes:

- `SurveyStartView`
- `SurveyAnswerView`
- `SurveyResultsView`

Endpoints:

- `POST /api/v1/ai/survey/start/`
- `POST /api/v1/ai/survey/answer/`
- `GET /api/v1/ai/survey//results/`

Purpose:

- start, answer, and retrieve the AI matching survey results

3. Architecture, routes, and logic flow map

3.1 Request lifecycle

The project follows a common request lifecycle:

1. URL is resolved by Django routing.
2. The relevant view class receives the HTTP request.
3. The view calls serializer validation and/or selectors.
4. Services execute business logic.
5. The database is queried or updated.
6. Serialized data is returned to the client.
7. A custom renderer formats the JSON response.

8. Background tasks may run asynchronously for notification or analytics work.

3.2 Core API route map

Buyer routes

- GET /health/
- GET /api/v1/auth/challenge/
- POST /api/v1/auth/send-otp/
- POST /api/v1/auth/verify-otp/
- POST /api/v1/auth/token/refresh/
- POST /api/v1/auth/logout/
- GET /api/v1/account/me/
- PATCH /api/v1/account/me/
- GET /api/v1/properties/
- GET /api/v1/properties//
- GET /api/v1/search/
- GET /api/v1/favorites/
- POST /api/v1/favorites//toggle/
- POST /api/v1/inquiries/
- GET /api/v1/inquiries/my/
- GET /api/v1/inquiries//
- POST /api/v1/contact/lead/
- POST /api/v1/ai/survey/start/
- POST /api/v1/ai/survey/answer/
- GET /api/v1/ai/survey//results/

Developer routes

- POST /api/v1/developer/login/
- POST /api/v1/developer/register/
- GET /api/v1/developer/account/
- PUT /api/v1/developer/account/
- DELETE /api/v1/developer/account/
- GET /api/v1/developer/team/
- POST /api/v1/developer/team/invite/
- PUT /api/v1/developer/team//
- DELETE /api/v1/developer/team//
- GET /api/v1/developer/projects/
- POST /api/v1/developer/projects/
- GET /api/v1/developer/projects//
- PUT /api/v1/developer/projects//
- DELETE /api/v1/developer/projects//
- GET /api/v1/developer/projects//stats/

- GET /api/v1/developer/dashboard/overview/
- GET /api/v1/developer/leads/
- GET /api/v1/developer/leads/export/

Supporting routes

- GET /api/v1/locations/
- GET /api/docs/schema/
- GET /api/docs/swagger-ui/
- GET /api/docs/redoc/

3.3 Data and dependency relationships

The central data relationships are:

- User owns favorites and inquiries
- Property is the main listing entity
- Favorite links User to Property
- Inquiry links buyer contact to a specific listing
- DeveloperAccount owns projects and team members
- Project is separate from property data but related to developer management
- MarketingLead captures external interest from campaigns
- SurveySession and SurveyAnswer tie user behavior to AI matching results

3.4 Cross-app logic patterns

The codebase repeats a general structure in each domain:

- models.py defines the persistent data
- selectors.py reads data with filtering
- services.py changes state or performs logic
- serializers.py validates or converts output
- views.py exposes the route
- urls.py registers the route
- tests/ verifies behavior

This makes the codebase understandable and consistent.

3.5 Security and validation patterns

The backend uses a layered validation model:

- serializer validation ensures API payload correctness
- model validation catches invalid state transitions
- custom validators fix phone and birthday logic
- ownership permissions restrict access to user-owned objects
- developer permission classes restrict internal operations
- JWT tokens secure user and developer API access

3.6 Async and background work

The project uses Celery-backed background tasks for work that should not block the API response, including:

- inquiry notification dispatch
- search pricing median refresh
- other scheduled background calculations

This is a strong pattern for real-world property marketplace backends because it keeps user requests fast while still processing downstream tasks.

3.7 Strengths of the architecture

The design is strong because:

- each app is domain-focused
- shared logic is centralized in common/
- business rules are in services, not mixed into views
- API responses are consistent and easy to consume
- tests cover the main flows and protect regressions
- background jobs reduce request-time complexity

3.8 Possible improvement areas

Possible future enhancements:

- stricter file/media validation for uploads
- more explicit role-based access enforcement across all dev endpoints
- more standardized API error specification across all modules
- deeper analytics layer for leads, conversion, and search behavior

4. Summary

This project is a complete Django REST backend for a real-estate ecosystem. It combines buyer functionality, developer operations, search logic, AI matching, and background tasks in a modular app structure. The code is organized around a clear service/selector/view pattern and is designed to be extensible and maintainable.

The most important idea is this:

The backend is not just a CRUD API. It is a real marketplace platform with user auth, listing discovery, search ranking, lead capture, developer management, AI recommendation flow, and analytics layered into one consistent system.